Helix OTA — Challenges + HelixQA + Anti-Bluff Audit (2026-06-23)

Revision: 1 **Last modified:** 2026-06-23T13:10:00Z **Method:** on-disk inspection at HEAD; FACT-with-citation or honest gap (§11.4.6). NO commit by the research stream.

Part 1 — Challenges + HelixQA

Challenges (`submodules/challenges/`, vasic-digital/Challenges). Generic reusable Go module: `pkg/challenge` (Challenge interface + BaseChallenge + ProgressReporter liveness), `pkg/registry` (topological dep ordering), `pkg/runner` (seq/parallel/pipeline + liveness kill), `pkg/assertion` (16 evaluators), `pkg/bank` (JSON/YAML banks), `pkg/userflow` (8 adapters / 21 impls: Playwright·ADB·Tauri·HTTP·gRPC·WS·Gradle·Cargo·npm, incl. recorded+video variants), `pkg/infra` (containers bridge), `cmd/userflow-runner`, `lib/anti_bluff.sh` (`ab_evidence_token` per-run UUID defeating stale-cache, `ab_assert_delta`, `ab_assert_kernel_value`). 28 example banks. **Run:** load a bank via `pkg/bank` → `runner.RunAll`, or shell-bank dispatch (bank entry names a real `dispatch_command` + `evidence_artifact`).

HelixQA (`submodules/helixqa/`, HelixDevelopment/HelixQA). Go QA orchestration engine composing Challenges+Containers: `pkg/orchestrator`, `pkg/testbank` (`dispatch.go` = canonical dispatch+evidence-ledger), `pkg/detector` (real-time crash/ANR), `pkg/validator`, `pkg/evidence`, `pkg/ticket`, `pkg/reporter`. ~25 vision/capture binaries in `cmd/` (`helixqa-text` OCR, omniparser, uitars, lpips, recvalidate, recording-analyzer, x11grab/kmsgrab, axtree) — the §11.4.107/117/137/160/163 vision+media toolchain. Ships `docker-compose.stack.yml` + `pkg/infra` (§11.4.76/161). **Run:** `cmd/helixqa` against the live system → orchestrator dispatches, detector watches, evidence collected, reporter emits MD/HTML/JSON + tickets; PASS requires positive captured evidence.

helix_ota existing coverage. `tools/helixqa/banks/helix_ota.yaml` (rev 4): 15 `HOTA-*` challenges (AUTH-LOGIN+401, DEVICE-REGISTER, GROUP-LIFECYCLE, AUDIT-TRAIL, TELEMETRY-OVERVIEW, ROLLOUT-ROUTE-GATES, PIPELINE-SIGNED, RECALL-LIFECYCLE, SECURITY-PROBES[-EXTENDED], FILTERS-PAGINATION, AB-VIRT-BOOT/SLOT-SWITCH/ROLLBACK, RK3588-CONTROLPLANE) + HELIXTRACK-001..003 + HOTA-CF-TIER2-AB. `tools/helixqa/run_bank.sh` machine-runs with GATE 1 (dispatch-exit-0) + GATE 2 (evidence-ledger non-empty `evidence_artifact`) + `--self-test`. Real dispatch in `tests/e2e/*` (black-box curl+jq vs live ota-server), `tests/emulator/*`, `tests/security/*`, `tests/helixqa/*` (Playwright).

Gaps: signed-artifact path often SKIPs (need a deterministic signed fixture + a **NEGATIVE bad-signature / request-supplied-key-ignored** challenge for the `resolvePublicKey` trust boundary); no telemetry-schema-validation challenge; rollout staged-progression+halt-on-failure thin; no §11.4.85 chaos challenges (malformed/oversized artifact, replay, concurrent rollout); UI recordings not bridged to HelixQA vision. **The bank is shell-dispatch only — the Go `cmd/helixqa` orchestrator + vision binaries are NOT yet wired against the OTA server.**

Part 2 — Anti-bluff audit

Bluff-proof (paired §1.1 mutation makes them FAIL) — 2: `run_bank.sh` GATE 1+2 (verified `--self-test` builds missing-evidence→FAIL + real-evidence→PASS); `test_constitution_inheritance.sh`. **Real-but-unpaired — 1:** `CM-COVERAGE-MINIMUM` (≥60% go test, no mutation companion). **Partial:** `challenge_operational.sh` (negative controls, no registered mutation); `tests/regression/run_all.sh` guards (RED-on-broken but not all paired); §11.4.166 semgrep (wired in `commit_all.sh` but **fail-opens silently when not on PATH**, unpaired). **NOT bluff-proof — 14:** `tests/pre_build_verification.sh` (80 lines) — 14 gates are bare `grep -qF '11.4.N'` anchor-presence checks. The functional gates `CM-FEATURE-STATUS-VIDEO-CONFIRMED`, `CM-MEDIA-VALIDATION-PIPELINE`, `CM-SEMGREP-WIRED`, `CM-VISION-VERIFIED-RECORDING-BRIDGE`, `CM-INDEPENDENT-VERIFICATION-AGENT` are NOT implemented (only their propagation counterparts). **§11.4.69 FACT:** `ab_pass_with_evidence` / `ab_skip_with_reason` exist only in constitution docs — **no helix_ota shell helper lib implements them**; e2e scripts use a local `pass`/`fail` idiom; evidence-path discipline is enforced at the bank-runner layer, not per-assertion.

Anti-bluff gap plan → "anti-bluff everywhere"

- **AB-G1** implement `tests/lib/anti_bluff.sh` (`ab_pass_with_evidence <desc> <path>` refuses PASS unless path exists+non-empty; `ab_skip_with_reason`); route every `tests/` PASS through it = mechanical per-assertion no-evidence-no-PASS.
- **AB-G2** one registered `tests/meta/meta_test_<gate>.sh` per functional gate (template = `run_bank.sh --self-test`): mutate→assert FAIL→restore→assert PASS; `tests/meta/run_all.sh` into pre-build.
- **AB-G3** replace the 14 presence-greps with functional gates (or pair each with an anchor-strip mutation); implement `CM-SEMGREP-WIRED` (no fail-open), the media/vision-bridge gates, the §11.4.165 independent-verifier.
- **AB-G4** close OTA challenge holes: bad-signature-rejected + request-key-ignored, telemetry-schema, rollout-staged+halt, §11.4.85 chaos — each with real dispatch + evidence artifact + paired mutation.
- **AB-G5** bridge the Go `cmd/helixqa` orchestrator (native crash/ANR/step-validation/evidence) + feed UI recordings through HelixQA vision for read-the-screen PASS (§11.4.27/§11.4.160).

Honest net (§11.4.6): the anti-bluff *pattern* is correct and present, but applied to only ~2–3 of ~17 pre-build gates; the per-assertion evidence helper is unimplemented in helix_ota; the HelixQA Go orchestrator + vision toolchain are uninstalled against the OTA system. **AB-G1 + AB-G2 are the highest-leverage steps** from "anti-bluff at the bank boundary" to "anti-bluff everywhere".